23장 구조체와 사용자 정의자료형2

차례

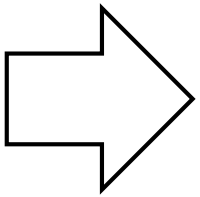
1. 구조체정의와 typedef 선언
2. 함수로의 구조체 변수 전달과 반환
3. 구조체의 유용함에 대한 논의와 중첩구조체
4. 공용체의 정의와 의미
5. 열거형의 정의와 의미



구조체를 정의하는 이유?

- 관련 있는 데이터를 하나로 묶을 수 있는 자료형의 정의가능
- 데이터의 표현 및 관리가 쉬워지고 생산성이 향상 -> 대규모의 코드 작성에서 필수적인 기능

```
int number;
char name[10];
double grade;
```



```
struct student {
        int number;
        char name[10];
        double grade;
};
```



예제 1


```c
#include <stdio.h>
typedef struct student
{
	char name[20];		// 학생 이름
	char stdnum[20];	// 학생 고유번호
	char school[20];	// 학교 이름
	char major[20];		// 선택 전공
	int year;		// 학년
} Student;
void ShowStudentInfo(Student* sptr);
int main(void)
{
	Student arr[7];
	int i;
	for (i = 0; i < 7; i++)
	{
		printf("이름: ");  scanf("%s", arr[i].name);
		printf("번호: ");  scanf("%s", arr[i].stdnum);
		printf("학교: ");  scanf("%s", arr[i].school);
		printf("전공: ");  scanf("%s", arr[i].major);
		printf("학년: ");  scanf("%d", &arr[i].year);
	}
```

```
이름: 홍길동 <엔터>
번호: 202301 <엔터>
학교: 군산대 <엔터>
전공: 소프트웨어 <엔터>
학년: 1 <엔터>
...
이름: 홍길동
번호: 202301
학교: 군산대
전공: 소프트웨어
학년: 1
...
```



예제 1

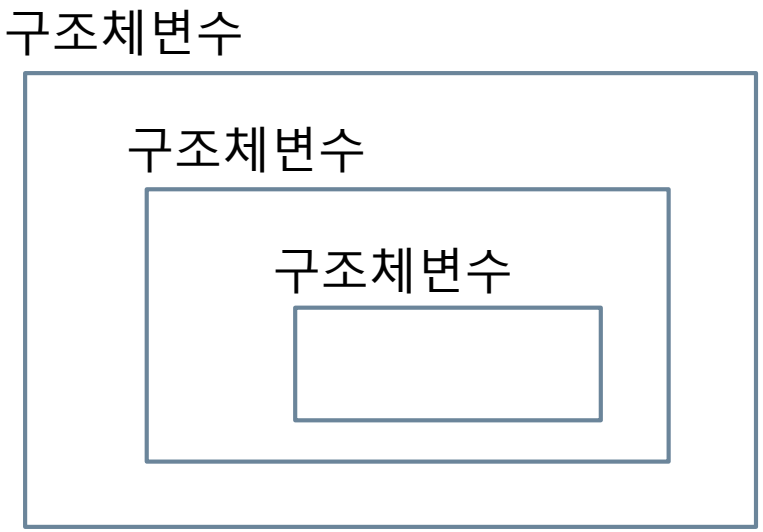

```
        for (i = 0; i < 7; i++) ShowStudentInfo(&arr[i]);
        return 0;
}
void ShowStudentInfo(Student* sptr)
{
        printf("학생 이름: %s ₩n", sptr->name);
        printf("학생 고유번호: %s ₩n", sptr->stdnum);
        printf("학교 이름: %s ₩n", sptr->school);
        printf("선택 전공: %s ₩n", sptr->major);
        printf("학년: %d ₩n", sptr->year);
}
```

```
이름: 홍길동 <엔터>
번호: 202301 <엔터>
학교: 군산대 <엔터>
전공: 소프트웨어 <엔터>
학년: 1 <엔터>
...
이름: 홍길동
번호: 202301
학교: 군산대
전공: 소프트웨어
학년: 1
...
```



중첩된 구조체

- 구조체변수도 구조체의 멤버로 선언가능
- 구조체안에 구조체 변수가 멤버인 경우 -> 중첩 구조체





예제2


```
#include <stdio.h>
typedef struct point
{
	int xpos;
	int ypos;
} Point;
typedef struct circle
{
	Point cen;
	double rad;
} Circle;
void ShowCircleInfo(Circle* cptr);
```

```
[1, 0]
radius: 3.5

[2, 4]
radius: 3.9
```



예제2


```
int main(void)
{
	Circle c1 = { {1,0}, 3.5 };
	Circle c2 = { 2, 4, 3.9 };
	ShowCircleInfo(&c1);
	ShowCircleInfo(&c2);
	return 0;
}
void ShowCircleInfo(Circle* cptr)
{
	printf("[%d, %d] ₩n", (cptr->cen).xpos, (cptr->cen).ypos);
	printf("radius: %g ₩n₩n", cptr->rad);
}
```

```
[1, 0]
radius: 3.5

[2, 4]
radius: 3.9
```



공용체(union)

- 정의 방법에 있어서의 차이는 키워드 struct -> union
- 공용체는 멤버변수가 메모리공간을 공유하는 방식임

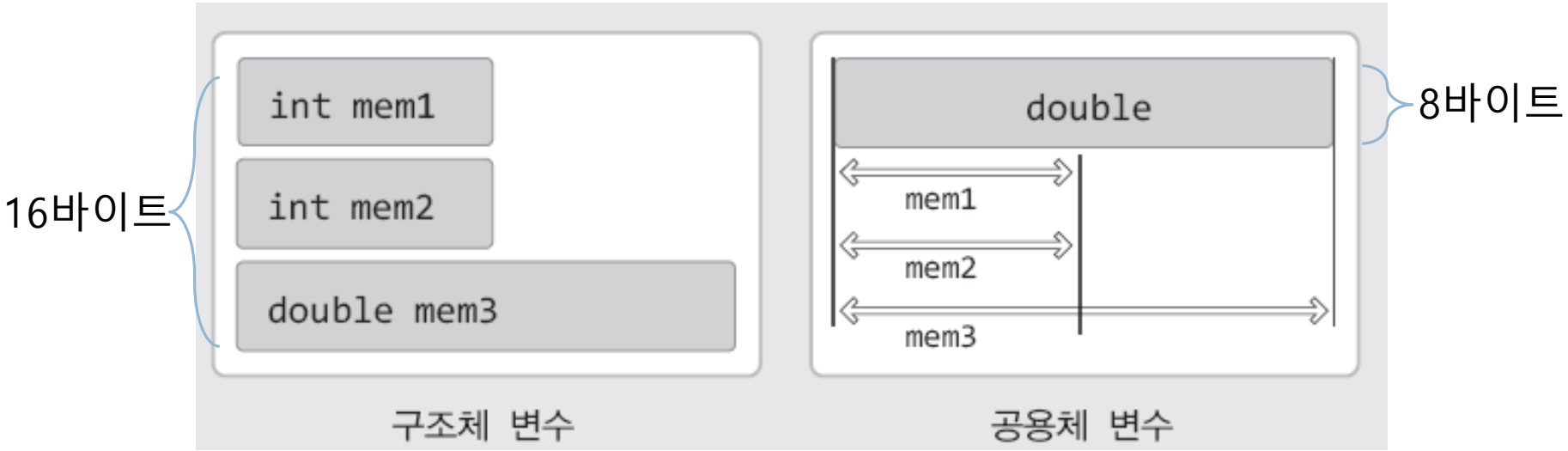
```
typedef struct sbox
{
    int mem1;
    int mem2;
    double mem3;
} SBox;
```

```
typedef union ubox
{
    int mem1;
    int mem2;
    double mem3;
} UBox;
```



공용체(union)

- 구조체를 구성하는 멤버는 각각 별도의 공간에 할당되지만 공용체의 멤버는 **크기가 가장 큰 멤버의 변수만 하나 할당**되고 나머지는 그 공간을 공유함





예제3


```
#include <stdio.h>
typedef struct sbox    // 구조체 sbox의 정의
{
      int mem1;
      int mem2;
      double mem3;
} SBox;

typedef union ubox    // 공용체 ubox의 정의
{
      int mem1;
      int mem2;
      double mem3;
} UBox;
```

```
E9FEFF858 E9FEFF85C E9FEFF860
E9FEFF888 E9FEFF888 E9FEFF888
16 8
```



예제3


```
int main(void){
        SBox sbx;
        UBox ubx;
        printf("%p %p %p ₩n", &sbx.mem1, &sbx.mem2, &sbx.mem3);
        printf("%p %p %p ₩n", &ubx.mem1, &ubx.mem2, &ubx.mem3);
        printf("%d %d ₩n", sizeof(SBox), sizeof(UBox));
        return 0;

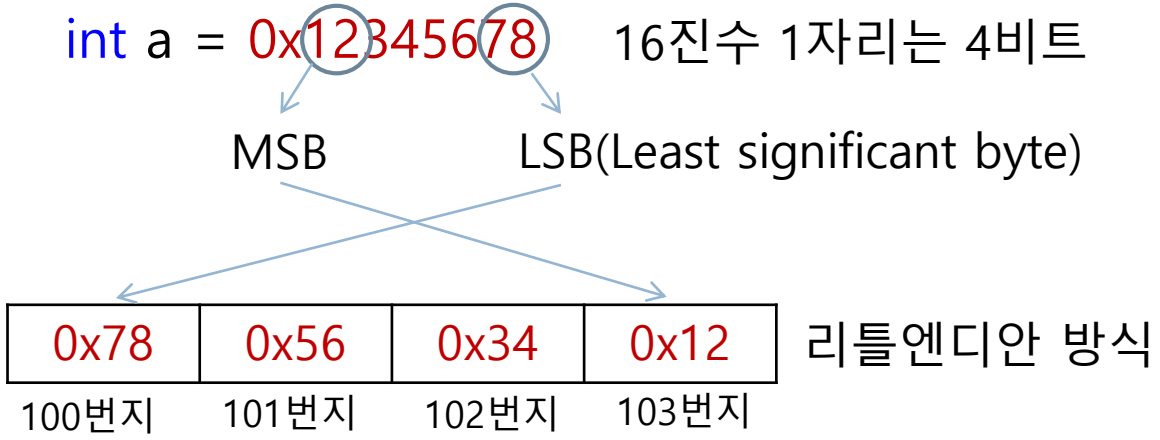
}
```

```
E9FEFF858 E9FEFF85C E9FEFF860
E9FEFF888 E9FEFF888 E9FEFF888
16 8
```



리틀엔디안(little endian)

- 데이터를 바이트 단위로 나누어서 메모리에 저장함
- 리틀엔디안 : 상위 바이트(MSB, Most significant byte)를 높은(큰) 주소에 저장하는 방식, Intel CPU에서 사용하는 방식
- 빅엔디안 : 상위 바이트(MSB)를 낮은(작은) 주소에 저장하는 방식, 네트워크 통신시 사용

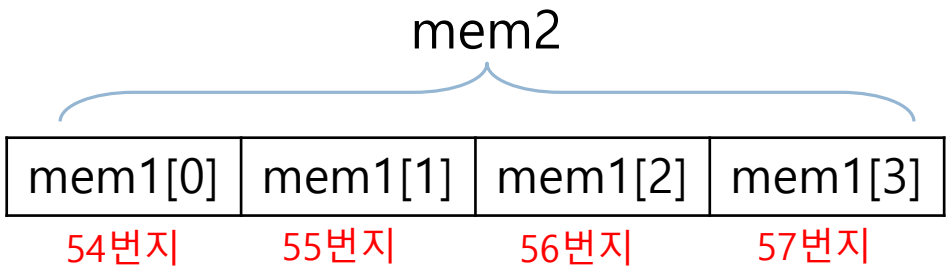




예제 4


```c
#include <stdio.h>
typedef union ubox     // 공용체 ubox의 정의
{
      char mem1[4];
      int mem2;
} UBox;
```

```
12345678
000000FA795FFA54, 78
000000FA795FFA55, 56
000000FA795FFA56, 34
000000FA795FFA57, 12
```





예제 4


```
int main(void)
{
      UBox ubx;     // 8바이트 메모리 할당
      ubx.mem2 = 0x12345678;
      printf("%x₩n", ubx.mem2);
      printf("%p,%x₩n", &ubx.mem1[0], ubx.mem1[0]);
      printf("%p,%x₩n", &ubx.mem1[1], ubx.mem1[1]);
      printf("%p,%x₩n", &ubx.mem1[2], ubx.mem1[2]);
      printf("%p,%x₩n", &ubx.mem1[3], ubx.mem1[3]);
      return 0;
}
```

```
12345678
000000FA795FFA54, 78
000000FA795FFA55, 56
000000FA795FFA56, 34
000000FA795FFA57, 12
```

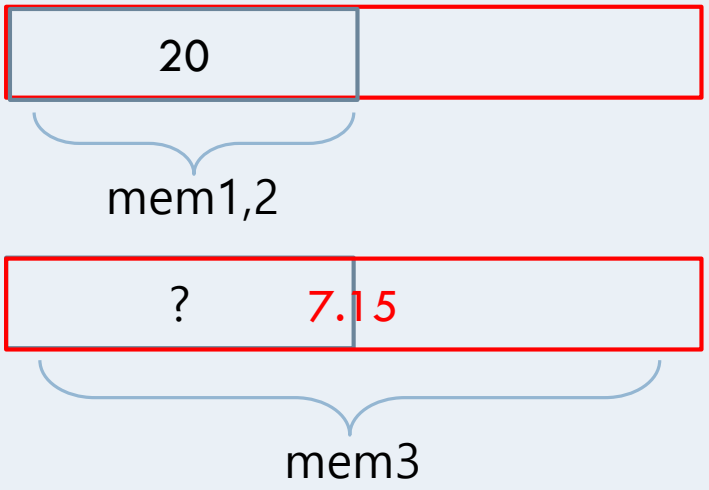
0x78	0x56	0x34	0x12
54번지	55번지	56번지	57번지



예제 5

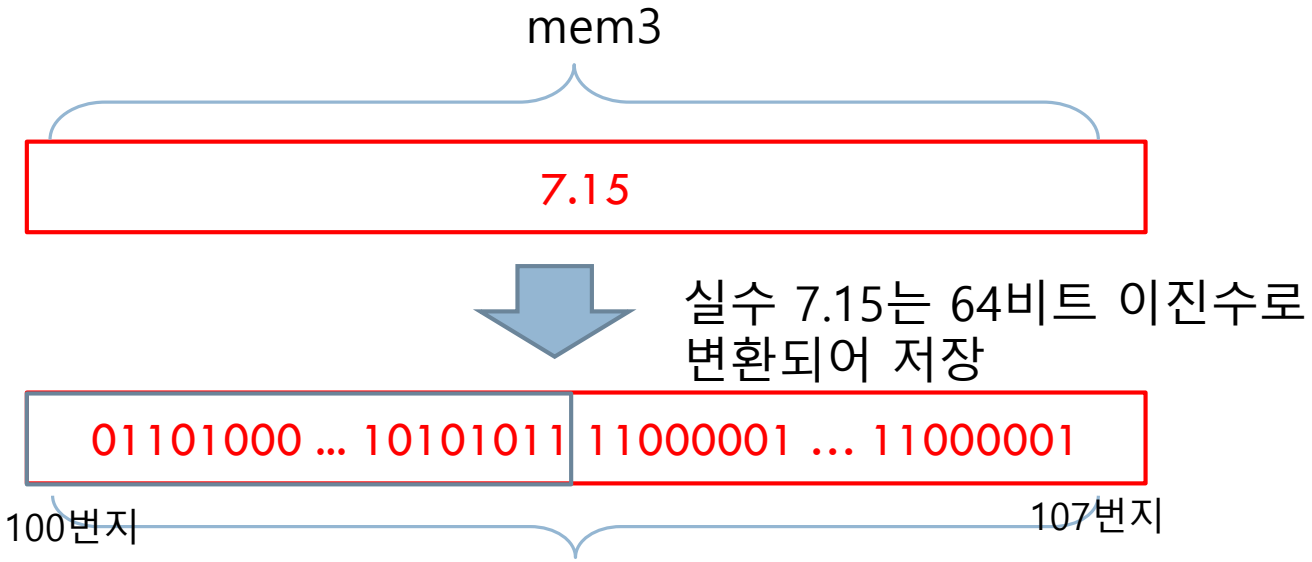

```c
#include <stdio.h>
typedef union ubox    // 공용체 ubox의 정의
{
      int mem1;
      int mem2;
      double mem3;
} UBox;
int main(void)
{
      UBox ubx;    // 8바이트 메모리 할당
      ubx.mem1 = 20;
      printf("%d ₩n", ubx.mem2);
      ubx.mem3 = 7.15;
      printf("%d ₩n", ubx.mem1);
      printf("%d ₩n", ubx.mem2);
      printf("%g ₩n", ubx.mem3);
      return 0;
}
```

```
20
-1717986918
-1717986918
7.15
```





예제 5

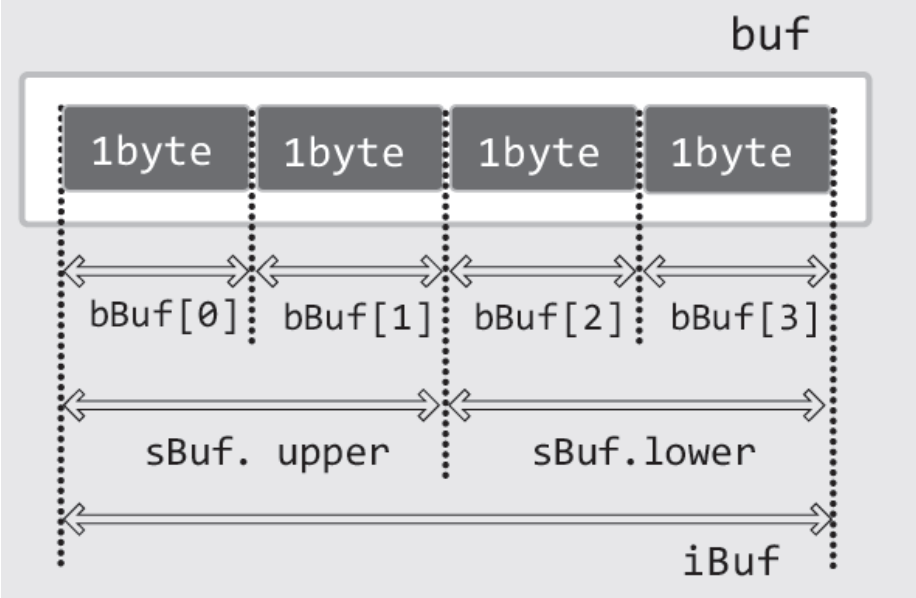

mem3
7.15
실수 7.15는 64비트 이진수로
변환되어 저장
01101000 … 10101011
11000001 … 11000001
100번지
107번지
mem1,2->하위 4바이트에 해당하는 값



공용체(union)의 유용함

- 하나의 메모리 공간을 2개 이상의 방식으로 접근 가능함

```
typedef struct dbshort
{
        unsigned short upper;
        unsigned short lower;
} DBShort;
typedef union rdbuf
{
        int iBuf;
        char bBuf[4];
        DBShort sBuf;
} RDBuf;
```





예제 5


```c
#include <stdio.h>
typedef struct dbshort
{
	unsigned short upper;
	unsigned short lower;
} DBShort;
typedef union rdbuf
{
	int iBuf;
	char bBuf[4];
	DBShort sBuf;
} RDBuf;
```

```
정수 입력: 1145258561
상위 2바이트: 16961
하위 2바이트: 17475
상위 1바이트 아스키 코드: A
하위 1바이트 아스키 코드: D
```



예제 5


```
int main(void)
{
        RDBuf buf;
        printf("정수 입력: ");
        scanf("%d", &(buf.iBuf));

        printf("상위 2바이트: %u ₩n", buf.sBuf.upper);
        printf("하위 2바이트: %u ₩n", buf.sBuf.lower);
        printf("상위 1바이트 아스키 코드: %c ₩n", buf.bBuf[0]);
        printf("하위 1바이트 아스키 코드: %c ₩n", buf.bBuf[3]);
        return 0;
}
```

```
정수 입력: 1145258561
상위 2바이트: 16961
하위 2바이트: 17475
상위 1바이트 아스키 코드: A
하위 1바이트 아스키 코드: D
```



열거형 필요성

- 기본자료형은 자료형이 가질 수 있는 값이 정해져 있음

자료형		크기(바이트)	데이터 범위
정수형	short	2	-32,768 ~ 32,767
	int	4	-2,147,483,648 ~ 2,147,483,647
	long long	8	–9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
문자형	char	1	-128 ~ 127
실수형	float	4	$-3.4\times10^{38} \sim -1.2\times10^{-38}$, 0, $1.2\times10^{-38} \sim 3.4\times10^{38}$
	double	8	$-1.8\times10^{308} \sim -2.2\times10^{-308}$, 0, $2.2\times10^{-308} \sim 1.8\times10^{308}$



열거형(enumeration)

- 자료형(변수)이 가질 수 있는 값들을 원하는 대로 정의할 수 있는 자료형 -> 구조체나 공용체는 멤버변수의 자료형만 정의 가능
- 자료형(변수)이 가질 수 있는 값을 기호형태의 상수(**열거형 상수**)로 모아 놓은 것
- 요일을 저장하는 자료형 -> {일요일, 월요일, 화요일, 수요일, 목요일, 금요일, 토요일 } 중의 하나의 값만 가지도록 정의가능

```
enum days  { SUN, MON, TUE, WED, THU, FRI, SAT };
```

자료형의 이름　　자료형이 가질 수 있는 값들->**열거형 상수**



열거형 정의 및 변수 선언

- 열거형 정의 -> 자료형 정의

```
enum levels { LOW, MEDIUM, HIGH };
```

키워드 열거형 이름 열거형 상수를 나열

- 열거형 변수 선언

```
enum  levels  english;          // 열거형 변수 선언
english = LOW;                  // 변수에 열거형 상수들 중 대입
```



열거형 상수의 값이 결정되는 방식

- 열거형 상수값을 명시하지 않으면 0부터 1씩 증가하면서 할당
- 할당된 값이 중복되면 에러 발생 -> 자동으로 중복정의를 방지
- typedef 선언으로 자료형을 재정의하여 사용함

```
enum color { RED, BLUE, WHITE, BLACK };
enum color { RED=0, BLUE=1, WHITE=2, BLACK=3 };
enum color { RED=3, BLUE, WHITE=6, BLACK };
enum color { RED=3, BLUE=4, WHITE=6, BLACK=7 };
typedef enum color { RED, BLUE, WHITE, BLACK } COLOR;
```



열거형 예


```
enum days  { SUN, MON, TUE, WED, THU, FRI, SAT };
enum months { JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV,
DEC };
enum major { COMMUNICATION, COMPUTER, ELECTRIC, ELECTRONICS };
enum component { MAIN_BOARD, CPU, GRAPHIC_CARD, DISK, MEMORY };
enum levels {LOW=1, MEDIUM, HIGH };
typedef enum days  { SUN, MON, TUE, WED, THU, FRI, SAT } DAYS;
```



열거형과 다른방법 비교

정수 사용	기호 상수	열거형

switch(code) {
case 1:
printf("LCD TV₩n");
break;
case 2:
printf("PDP TV₩n");
break;
}
	#define LCD 1
	#define PDP 2
	
	switch(code) {
	case LCD:
	printf("LCD TV₩n");
	break;
	case PDP:
	printf("PDP TV₩n");
	break;
	}
		enum tvtype { LCD, PDP };
		enum tvtype code;
		
		switch(code) {
		case LCD:
		printf("LCD TV₩n");
		break;
		case PDP:
		printf("PDP TV₩n");
		break;
		}

숫자의 의미를 기억해야 한다.	기호 상수를 작성할 때 오류를 저지를 수 있다 (같은 값으로 정의할 수 있다)	컴파일러가 기호상수 값이 중복되지 않도록 자동 할당한다.



예제6 : 열거형을 사용하지 않는 경우


```
#include <stdio.h>
void Sound(int sy);
int main(void)
{
	int tone;
	for (tone = 1; tone <= 7; tone += 1)
		Sound(tone);
	return 0;
}
void Sound(int sy)
{
	switch (sy)
	{
	case 1: puts("도는 하얀 도라지 ♪"); break;
	case 2: puts("레는 둥근 레코드 ♩"); break;
	case 3: puts("미는 파란 미나리 ♩♪"); break;
	case 4: puts("파는 예쁜 파랑새 ♪♭"); break;
	case 5: puts("솔은 작은 솔방울 ♩♪♪"); break;
	case 6: puts("라는 라디오고요~ ♪♩♭♩"); break;
	case 7: puts("시는 졸졸 시냇물 ♩♭♩♪"); break;
	}
}
```

```
// 음계를 숫자로 표현 : Do->1, Re->2, Mi->3,
// Fa->4, So->5, La->6, Ti->7으로 정의
```



예제6 : 열거형을 사용하는 경우


```
#include <stdio.h>
typedef enum syllable
{
	Do, Re, Mi, Fa, So, La, Ti
} Syllable;
void Sound(Syllable sy);
int main(void)
{
	Syllable tone;
	for (tone = Do; tone <= Ti; tone += 1)
		Sound(tone);
	return 0;
}
```



예제6 : 열거형을 사용하는 경우


```
void Sound(Syllable sy)
{
	switch (sy)
	{
	case Do: puts("도는 하얀 도라지 ♪"); break;
	case Re: puts("레는 둥근 레코드 ♩"); break;
	case Mi: puts("미는 파란 미나리 ♩♪"); break;
	case Fa: puts("파는 예쁜 파랑새 ♪♭"); break;
	case So: puts("솔은 작은 솔방울 ♩♪♪"); break;
	case La: puts("라는 라디오고요~ ♪♩♭♩"); break;
	case Ti: puts("시는 졸졸 시냇물 ♩♭♩♪"); break;
	}
}
```

```
도는 하얀 도라지 ♪
레는 둥근 레코드 ♩
미는 파란 미나리 ♩♪
파는 예쁜 파랑새 ♪♭
솔은 작은 솔방울 ♩♪♪
라는 라디오고요~ ♪♩♭♩
시는 졸졸 시냇물 ♩♭♩♪
```



열거형의 유용함

- Syllable이라는 이름의 자료형 안에서 음계에 관련 있는 상수들을 모두 묶어서 정의가능 -> 프로그램의 가독성 높임
- 변수의 선언이 목적이 아닌 상수의 선언이 목적인 경우 열거형의 이름과 typedef 선언을 생략하기도 함

```
typedef enum syllable
{
    Do, Re, Mi, Fa, So, La, Ti
} Syllable;
```

```
enum { Do, Re, Mi, Fa, So, La, Ti };
```



실습과제 1

- 구조체가 왜 필요한지 설명하시오.
- 공용체가 왜 필요한지 설명하시오.
- 열거형이 왜 필요한지 설명하시오.



실습과제2

- 학생이름,학번,국어점수,영어점수,수학점수, 평균점수로 구성되는 구조체를 정의하고 3명의 정보를 입력 받는다. 평균점수는 국어,영어,수학점수를 입력 받아 계산하여 저장하고 3명중 최우수 학생의 정보를 출력하시오.

```
이름: 홍길동<엔터>
학번: 2301<엔터>
국어 : 90<엔터>
영어 : 80<엔터>
수학 : 70<엔터>

....
<최우수 학생정보>
김우수 2303 90 90 90 90.00
```



실습과제3

- 공용체를 이용하여 정수를 16진수로 입력 받고(서식문자 %x) 각 바이트를 아래처럼 출력해주는(서식문자 %x) 코드를 작성하시오.

```
정수를 입력하시오(16진수): 11223344 <엔터>
최상위(4번째) 바이트(16진수) : 11
3번째 바이트(16진수) : 22
2번째 바이트(16진수) : 33
최하위(1번째) 바이트(16진수) : 44
```



실습과제 4

- 아래코드를 열거형을 이용하여 가독성이 높은 코드로 수정해보시오.

```
#include <stdio.h>
int main(void)
{
    int direction=10;

    switch (direction)
    {
    case 10: printf("현재방향: 위"); break;
    case 20: printf("현재방향: 아래"); break;
    case 30: printf("현재방향: 왼쪽"); break;
    case 40: printf("현재방향: 오른쪽"); break;
    }
    return 0;
}
```



과제제출방법

- 아래 주소의 작성방법대로 github에 제출할 것
  - https://github.com/2sungryul/cprogramming/tree/main
- 기한 : 이클래스 과제 게시판의 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목,날짜,작성자(학번,이름)를 작성할 것

```
// ********************************************
//    제   목 : 정수 4개의 평균을 구하는 프로그램
//    날   짜 : 2023년 9월10일
//    작성자 : 15010101 홍길동
// ********************************************

// 소스코드 작성
```